

Security Review Response

Answers to the architecture, CMS security and file-upload questions raised by ICON-INSTITUTE's website host.

Client	ICON-INSTITUTE GmbH & Co. KG Consulting Gruppe
Subject	Architecture, CMS security and file uploads
Reference	Addendum to the Technical Documentation & Technical Review Response, July 2026
Prepared by	Codexcape Solutions — Sohaima Rasheed
Date	9 August 2026

Every answer below was verified against the running system rather than from documentation. Where something is not yet in place, it is stated plainly and listed again in section 04.

01

ARCHITECTURE & TECHNOLOGY

Q What technology or framework is the website built with?

The public website is **React 19 with TypeScript**, compiled by Vite into plain static HTML, CSS and JavaScript. The CMS behind it is **FastAPI on Python 3.13** with **MongoDB** for storage.

There is **no PHP, no WordPress, no Laravel** anywhere in the stack. Serving a visitor a page involves no server-side rendering and no database query — the removal of that dynamic layer is one of the security objectives of the project.

Q Does the public website communicate directly with the CMS?

It reads from a **separate, read-only public interface** that returns published content only. Drafts, scheduled items and archived entries are not visible to it under any circumstances.

The public site holds no credentials, opens no session and has **no write path of any kind**. The single exception is the job-application form, covered in section 03.

Q Which components are publicly accessible from the Internet?

Of **86** endpoints in the system, **10** are reachable without authentication. All are read-only:

- Published news, jobs and projects — list and individual item
- Published page content, contact details and search-engine metadata
- Job applications — the one public submission point
- `/health` for monitoring, and `/media/<file>` for published images and documents

Everything else — content editing, users, media management, the audit log, backups and publishing — refuses anonymous callers. Verified directly:

GET	<code>/api/v1/news</code>	→ 401	GET	<code>/api/v1/audit</code>	→ 401
GET	<code>/api/v1/users</code>	→ 401	GET	<code>/api/v1/backups</code>	→ 401
GET	<code>/api/v1/media</code>	→ 401	POST	<code>/api/v1/publish/generate</code>	→ 401

The CMS service itself binds to `127.0.0.1` and is never exposed to the network directly; it is reached only through the web server in front of it.

02

CMS SECURITY

Q What CMS are you using, and what version?

• NOT A THIRD-PARTY CMS

This is a **purpose-built CMS written for ICON-INSTITUTE**, not a licensed or open-source product. That is the substance of the project: the WordPress installation, and the always-open automated upload channel feeding its Projects section, are both being removed.

The practical consequence is that there is no vendor CMS to be scanned for, no plugin marketplace, and none of the automated mass-exploitation traffic that follows a recognisable CMS fingerprint.

Q Is the CMS version currently supported by the vendor?

Not applicable in the usual sense — there is no vendor release cycle. What carries the equivalent risk is the set of libraries the system is built on, and each is on a current, supported release:

COMPONENT	VERSION IN USE	ROLE
Python	3.13.7	Runtime
FastAPI	0.141.1	Web framework
Starlette	1.5.0	HTTP layer
Pydantic	2.13.4	Input validation
PyMongo	4.17.0	Database driver
MongoDB	8.2	Database
PyJWT	2.13.0	Session tokens
Pillow	12.3.0	Image processing
argon2-cffi	25.1.0	Password hashing

The full list is eleven direct dependencies, all permissively licensed and pinned to exact versions in `requirements.txt`.

q How are security updates applied, and how quickly are critical vulnerabilities patched?

● **PROCESS TO BE AGREED**

Updating is mechanical: versions are pinned in one file, so an update is a version change, a rebuild and a redeploy, with the automated test suites run before release.

We should be candid that **an agreed monitoring and response process is not yet in place**, because it belongs with the maintenance agreement (§12 of the Review Response), which is still open. Our recommendation, which we can configure immediately on request:

- **Dependabot** on both repositories — raises a pull request per published advisory, at no cost
 - **pip-audit** in the build, so a known-vulnerable dependency fails the build rather than shipping
 - A written response target — we suggest **72 hours for critical, 14 days for high** — to be fixed in the maintenance agreement
-

q Are plugins, modules or extensions installed?

● **NONE**

There is **no plugin system**, so no third-party code can be added to the CMS at runtime — by an administrator or by an attacker who obtains an administrator account. All behaviour comes from the delivered source code and its eleven pinned dependencies.

This closes the single largest source of CMS compromise in practice, which is a vulnerable or abandoned plugin rather than the CMS core.

q How do you monitor vulnerabilities in CMS components?

Covered by the same answer as patching above: the tooling is straightforward to enable and we recommend doing so, but the monitoring cadence and response times are a maintenance-agreement decision rather than something we should set unilaterally.

Q Can the administration panel be restricted to VPN, corporate IP addresses or another trusted network?

● YES — AND WE RECOMMEND IT

The administrative surface is cleanly separated by address: `/admin` on the website and `/api/v1/*` on the CMS service. Both can be restricted at your web server by IP range or behind VPN **without affecting the public site at all**, because the public site reads from `/api/v1/public/*`, which would remain open.

Since the CMS service already listens only on the loopback address and is reached solely through your web server, that server is the correct and only enforcement point. We will supply the exact configuration block for your web server on request.

03

FILE UPLOADS

Q What file types are allowed?

A strict allow-list. Anything not named here is refused:

CONTEXT	PERMITTED TYPES	LIMITS
Media library — images	jpg, jpeg, png, webp, gif	25 MB
Media library — documents	pdf, doc, docx, xls, xlsx, ppt, pptx, zip, txt	25 MB
Media library — video	mp4, webm	25 MB
Job applications (public)	pdf, doc, docx, zip	10 MB

The public application form is deliberately the narrowest, since it is the only upload an unauthenticated visitor can perform. It accepts at most six files per application.

Q Are file extensions validated?

● YES

The extension is checked against the allow-list before anything is written to storage. A file that fails is rejected outright with a validation error and never reaches the disk or the database.

Q Are uploaded files renamed?

• YES, ALWAYS

The original name is reduced to lower-case letters, digits and hyphens, truncated, and given a random suffix:

```
"My Report (final).PDF" → my-report-final-a3f91b2c.pdf
```

The original name is kept only as a display label in the database, never as a path. This removes directory-traversal and name-collision attacks, and means a caller cannot predict or choose the stored path.

Q Are executable files blocked?

• YES

Executable and script types — `.php`, `.exe`, `.sh`, `.js`, `.html` and everything else outside the list above — are simply not permitted. Because the rule is an allow-list rather than a block-list, a type nobody anticipated is refused by default.

Beyond that, **every uploaded image is decoded and re-encoded** before storage. Anything hidden in the image data does not survive that process, which closes the polyglot-file route where a payload is disguised inside an otherwise valid picture.

Q Are uploaded files stored outside the web root where possible?

• YES — AND FOR MEDIA, BETTER THAN THAT

Media files are not on the filesystem at all. They are held inside the CMS database and streamed out by the application. There is therefore no directory anywhere that a web server could be misconfigured into executing, and no path an attacker could reach directly.

Every file is served with a fixed content type, `X-Content-Type-Options: nosniff` so the browser cannot be talked into treating a document as script, and anything not meant to be viewed in place is sent as a download.

ONE EXCEPTION, STATED FOR COMPLETENESS

Job-application CVs are still written to a directory on the server. That directory sits **outside the web root** and its contents are downloadable only through an authenticated CMS session with the correct permission — but it is the one upload path not yet moved into the database. We propose moving it for consistency and will do so on your confirmation.

04

OPEN POINTS

Three items from the answers above are not yet complete. None is a defect in the build; each needs a decision or a step on the hosting side.

ITEM	OWNER	NEXT STEP
Dependency vulnerability monitoring	Codexcape, on your approval	Enable Dependabot and pip-audit; agree response times in the maintenance agreement (§12).
HTTPS enforcement (HSTS)	Website host	Built and ready. It is switched on the moment the TLS certificate is active on the server.
Application CVs into the database	Codexcape, on your approval	Move the last upload path off the filesystem, matching the media library.

We would also encourage taking up the network restriction on `/admin` described in section 02. It costs nothing, changes no code, and removes the CMS sign-in page from public view entirely.